Spelf

A technical guide to the language, its compiler and the evidence behind it

Burak

Version 0.16, October 2026

Contents

Preface 4

1. Why another language 5

1.1 The failures that keep happening 5

1.2 What the existing tools cover 5

1.3 The position Spelf takes 5

2. A first look 7

2.1 A small program 7

2.2 From source to flight binary 7

2.3 The compiler in practice 8

3. Values that carry physics 9

3.1 Integers with ranges 9

3.2 Floating point, honestly 9

3.3 Units of measure 10

3.4 Reference frames and rotations 10

3.5 Time 11

3.6 Records, invariants and canonical forms 11

3.7 Fixed point 12

4. Sensors and persistent state 13

4.1 Suspect values 13

4.2 Provenance 13

4.3 Why this is a language feature and not a library 14

5. Rules specific to control software 15

5.1 Saturation and integrator windup 15

5.2 Redundancy and voting 15

5.3 Validated envelopes 16

5.4 Modes, halting and safe paths 16

6. Concurrency and timing 17

6.1 Tasks and protected objects 17

6.2 Schedulability 17

6.3 Stack depth 17

7. Memory and devices 18

7.1 No heap, no surprises 18

7.2 Resources with states 18

7.3 Memory regions . . . 19
8. What the compiler generates . . . 20
8.1 The C subset . . . 20
8.2 Targets and substrates . . . 20
8.3 Assurance levels . . . 20
9. How the compiler proves things . . . 21
9.1 Obligations . . . 21
9.2 Dead code . . . 21
9.3 The evidence bundle . . . 21
9.4 Why the solver budget is deterministic . . . 22
10. Evidence . . . 23
10.1 Method . . . 23
10.2 Twenty-six defects, six languages . . . 23
10.3 Complex systems in closed loop . . . 25
10.4 Cost . . . 28
10.5 Real bugs . . . 30
10.6 Is the compiler right? . . . 31
10.7 Threats to validity, and what we would like others to do . . . 32
11. Writing Spelf in practice . . . 33
11.1 Start from the declarations . . . 33
11.2 Reading a rejection . . . 33
11.3 Patterns that work . . . 33
11.4 Porting from C . . . 34
12. Limits . . . 35
Appendix A. Rule reference . . . 36
Appendix B. Grammar summary . . . 37
Appendix C. Reproducing the results . . . 38

Preface

This guide is for engineers who write or review flight software and want to know, in some detail, what Spelf is, how it is used, what it proves and what it does not. It assumes you have written embedded C or Ada and that you know roughly what a range check, an SMT solver and a rate-monotonic schedule are. It does not assume you have read anything else about Spelf.

The guide is long because the questions people ask about a new safety language are specific. "Is it as fast as C?" has a number. "Does it catch unit errors?" has a rule and a counterexample. "How do I know the compiler is right?" has five separate answers, each with a different strength. We have tried to give the number, the rule and the answer, and to say plainly where the evidence is thin.

Chapters 1 to 3 explain why the language exists and what it looks like. Chapters 4 to 8 go through the language feature by feature, with the rule each feature enforces. Chapter 9 explains how the compiler turns a program into proof obligations and C. Chapter 10 is the evaluation: what we measured, how, and what the results say. Chapter 11 is practical advice. Chapter 12 lists the limits. The appendices are reference material: every rule, the grammar, and the commands that reproduce the results.

1. Why another language

1.1 The failures that keep happening

If you read accident reports for space and aviation software, a pattern shows up quickly. The failures that end missions are rarely subtle. The Mars Climate Orbiter was lost in 1999 because one program produced impulse in pound-force seconds and another read it as newton seconds. Ariane 5 flight 501 was lost in 1996 because a 64-bit floating-point value was converted to a 16-bit integer that could not hold it; the code had been correct on Ariane 4, whose trajectory never produced such a value. Mars Polar Lander, the same year as the Orbiter, shut down its descent engines about 40 metres above the surface because the vibration of its legs deploying had been read as touchdown and remembered. Schiaparelli in 2016 trusted a saturated inertial measurement long enough to conclude it had already landed. Mars Pathfinder kept resetting on the surface because a low-priority task held a lock a high-priority task needed. Qantas flight 72 pitched down twice in 2008 because a filter let a second spike through exactly when its hold period expired.

None of these is an algorithmic mystery. In each case the program contained, somewhere, a fact that was true and that nobody had written down in a form a machine could check: this number is in newton seconds; this value can exceed 32767; this signal can be a transient; this measurement may be saturated; this lock is shared with a lower-priority task. The errors happened at the seams between modules, teams and years, exactly where human review is weakest.

1.2 What the existing tools cover

Flight software today is mostly written in C under a coding standard such as MISRA C or the JPL rules, sometimes in Ada, increasingly with SPARK for the parts that need proof, and in a few new projects in Rust. Each of these does something real.

MISRA C removes the most dangerous corners of C and makes code reviewable. It does not know what a unit is, and its checkers cannot tell a newton from a pound-force. Ada adds strong typing and range checks; a range violation raises an exception at run time instead of corrupting memory, which is a large improvement but still a run-time event, in flight. SPARK proves the absence of run-time errors statically, which is the strongest guarantee in common use, and it is the tool to beat. Rust removes memory unsafety and data races by construction and checks array bounds at run time.

What none of them does, out of the box, is reason about the physics and the provenance of a value. SPARK can prove that a conversion does not overflow; it cannot know that the value came from a sensor that may be saturated, or that it is expressed in the wrong reference frame, or that a boolean was set by a two-cycle transient. You can encode some of this with careful use of types and contracts, and good SPARK teams do. Our measurements in chapter 10 show where that works and where it does not: of 26 defect classes taken from real accidents, SPARK with its standard mechanisms catches 19 and misses 7. The seven are the ones about provenance, persistence and timing.

1.3 The position Spelf takes

Spelf takes three positions, and the rest of the design follows from them.

The first is that every fact a flight engineer knows about a value and that matters for safety should be part of its type: its range, its unit, its reference frame, whether it came from a sensor, whether it has been through a saturating limiter. Writing the fact once, at the declaration, is cheaper than checking it at every use, and it can be checked by a machine.

The second is that a run-time check in flight code is a failure of the development process, not a safety feature. If the compiler cannot prove that an index is in bounds, the answer is not to insert a check that will raise an exception during descent; it is to refuse the program and tell the engineer which input breaks it. Spelf therefore generates code with no run-time checks at all, and the generated C has no undefined behaviour because every operation that could have it has been proved safe.

The third is that the language has to be as fast as the C that is allowed to fly today, otherwise nobody will use it where it matters. Spelf compiles to C and lets mature C compilers do the code generation. Chapter 10 shows that on seven flight workloads and three flight processors, the result is within a fraction of a percent of hand-written flight C.

There is a fourth position, more a habit than a principle: every claim about the language should come with a test that would fail if the claim were false. This guide tries to follow that habit too.

2. A first look

2.1 A small program

Here is the core of a touchdown detector for a Mars lander. It is short, but it shows most of what is unusual about Spelf.

```
module Tutorial.Sensors

type Alt = Float32<m, 0.0..=2500.0>;

state armed: Latch = latch(persists_for = 5);
state touchdown: Latch = latch(persists_for = 3);

fn contact(r: Suspect<bool>) -> Int<0..=1> {
  match r {
    Valid(c) => if c { 1 } else { 0 },
    _ => 0,
  }
}

fn engines_off(alt: Suspect<Alt>, l1: Suspect<bool>, l2: Suspect<bool>, l3: Suspect<bool>) -> bool {
  match alt {
    Valid(h) => feed(armed, h < 40.0[m]),
    _ => feed(armed, false),
  }
  feed(touchdown, is_set(armed) && contact(l1) + contact(l2) + contact(l3) >= 2);
  is_set(touchdown)
}
```

`Float32<m, 0.0..=2500.0>` is a single-precision float in metres whose value is always between 0 and 2500. `Suspect<bool>` is a reading from a sensor: it can be `Valid`, `Saturated`, `Stale` or `Invalid`, and the program cannot use the boolean inside it without saying what happens in each case. `Latch` is a persistent decision that becomes true only after a declared number of consecutive true inputs and then stays true. The radar altitude arms the touchdown detector only after five consecutive valid readings below 40 m; two legs out of three must then report contact for three consecutive cycles.

If you rewrite this the way Mars Polar Lander's software effectively worked, with a plain boolean set the first time a leg reports contact, the compiler refuses it with rule R-404: a value that came from a sensor may not change a persistent decision except through a latch with a declared persistence. If you use the altitude value without checking its status, it refuses with R-402. Those two rules are the whole point of the example. The C, Rust and Ada versions of both mistakes compile without a warning from any tool we tried, and in closed-loop simulation the lander hits the ground at 55.6 m/s and 35.3 m/s respectively (section 10.3).

2.2 From source to flight binary

A Spelf program goes through these stages:

```
source.spelf -> parser -> type, unit, frame and mode checking -> proof obligations -> Z3
                                                                          |
                                                          program with every obligation proved
                                                                          |
                                                  undefined-behaviour-free C11 + evidence bundle (SMT-LIB)
                                                                          |
                                                          GCC / Clang for LEON3, RISC-V, ARM Cortex-R
```

The checker walks the program once and produces a proof obligation for every operation that could fail: an integer that could leave its range, a division whose divisor could be zero, an array index that could be out of bounds, a floating-point value that could leave its declared range or become infinite, a

narrowing that might not fit, a representation invariant that might not hold, a task set that might miss a deadline. Each obligation is a logical formula; Z3 either proves it, finds a counterexample, or gives up. A counterexample becomes an error message that names the input that breaks the rule. Giving up is also an error (rule R-602): an unproved obligation is never treated as proved.

When every obligation is proved, the code generator emits C11. The C uses only operations whose behaviour is defined for the values the proofs allow: integer widths are chosen from the proved ranges so no operation can overflow, and there are no pointers except to static storage, no heap and no recursion without a declared bound. The C is then compiled by an ordinary compiler for the target. Spelf does not trust the C compiler with anything that is not defined behaviour; it gives it only programs whose meaning the C standard pins down.

Alongside the C, `spelfc evidence` writes every proof obligation as a separate SMT-LIB file, so that a second solver (we use cvc5) can re-check them without running the Spelf compiler at all. This is meant for certification: the person who signs off does not have to trust our checker, only a standard solver and the translation of each obligation, which is readable.

2.3 The compiler in practice

The prototype compiler, `spelfc`, is written in Python and uses Z3 for proofs. It runs on any machine with Python 3.11. The commands are:

Command	What it does
`spelfc check f.spelf`	parse, type-check and prove every obligation; print a proof report
`spelfc build f.spelf -o f.c`	the same, then write C11 (`--freestanding` for targets without an operating system)
`spelfc run f.spelf`	build, compile for the host and run `main`
`spelfc interp f.spelf`	run `main` in the reference interpreter, with the semantic oracle on
`spelfc evidence f.spelf`	write the evidence bundle: one SMT-LIB file per obligation, plus a manifest
`spelfc stack --target leon3 f.spelf`	worst-case stack depth from the target assembly
`spelfc mem f.spelf`	memory regions: declared capacity against the real object layout
`spelfc ir f.spelf`	the checked program as JSON, for other tools

A proof report looks like this:

```
PROOF REPORT
  obligations: 40, all proved (Z3, rlimit=20000000, deterministic, floating-point model: ieee)
    R-407    1
    R-501    39
  note: assurance level: L2
```

The solver budget is a resource counter (`rlimit`), not a timeout, so the same program gives the same result on every machine. That matters more than it sounds: a proof that passes on a fast workstation and fails on the build server is not a proof anyone can certify.

3. Values that carry physics

3.1 Integers with ranges

Every integer type in Spelf has a range: `Int<0..=255>`, `Int<-32768..=32767>`, `Int<1..=65536>`. There is no plain `int`. Arithmetic is mathematical: `a + b` of two `Int<0..=255>` has type `Int<0..=510>`, computed exactly. When a value flows into a narrower type, through an assignment, a parameter or a return, the compiler must prove that it fits, under the path conditions that hold at that point. If it cannot, it shows a counterexample.

```
fn word(hi: Int<0..=255>, lo: Int<0..=255>) -> Int<0..=65535> {
  hi * 256 + lo
}
```

Here the compiler computes the range of `hi * 256 + lo` as 0 to 65535 and accepts the return. Change the return type to `Int<0..=65534>` and it refuses, with `hi = 255, lo = 255` as the witness.

Because ranges are known, the code generator chooses the C type from them. Arithmetic is done in `int32_t` when the operands and the result fit, otherwise `int64_t`; values stored in arrays, state and protected objects use the narrowest type that fits (`uint8_t`, `int16_t` and so on). Every C operation is done in a type wide enough for its operands and result, so overflow cannot happen in the generated code. This is not a convention the programmer has to follow; it falls out of the proofs.

Division truncates toward zero, as in C, and every `/` and `%` carries the obligation that the divisor is not zero. Integers in Spelf never wrap. If you want modular arithmetic, as for a sequence counter, you write the modulo explicitly; the 787 generator control unit counter overflow (a signed 32-bit centisecond counter that overflowed after 248 days) is one of the defect classes in the benchmark.

3.2 Floating point, honestly

Floats come in two widths, `Float` (IEEE-754 binary64) and `Float32` (binary32), and both are modelled exactly: the checker uses Z3's floating-point theory with round-to-nearest-even, so a property proved about a Spelf float holds for the C `double` or `float` that runs on the target, bit for bit. This is a deliberate choice. Most verification tools model floats as real numbers by default because real arithmetic is easier for solvers. A real-number model happily proves that `0.1 + 0.2 <= 0.3`, which is false in IEEE arithmetic. Spelf has a real-number mode (`--float-model real`) only for comparison and diagnosis; it is not accepted at assurance level L2 and above.

Exact floating-point reasoning is expensive, so the checker tries cheaper methods first. It starts with a linear abstraction of the expression, then a relaxed real model with error bounds that are themselves proved sound (rounding is monotone and preserves sign, and these facts are lemmas in the mechanised soundness proof), and only falls back to the full bit-precise model when the cheaper ones cannot decide. When even the full model runs out of budget, the checker searches deterministically for a concrete witness, evaluates it in exact IEEE arithmetic, and if it really violates the property, reports it under the original rule with the concrete values. If no witness is found the obligation stays unproved and the program is rejected.

A float type can carry a range, `Float32<rad/s, -2.0..=2.0>`, and then the range is an obligation like any other. A float without a range can hold any finite value, but most useful properties cannot be proved about it. In practice, flight code wants ranges on almost everything, because a range is how you prove that a filter output, a gain-scheduled product or an integrator stays finite.

The two widths do not mix silently. A binary64 value becomes binary32 only through `narrow<Float32<...>>(x)`, which is a rounding conversion the reader can see; literals adapt to the width of their context, so `0.44` next to a `Float32` is the binary32 constant `0.44f`.

3.3 Units of measure

Every float carries a unit. A unit in Spelf is a pair: a dimension vector over length, mass, time, current, angle and information, and a scale factor to SI. Two quantities can be added, compared or assigned to each other only if both the dimension and the scale agree. Dimension analysis alone would not have saved the Mars Climate Orbiter, because pound-force seconds and newton seconds have the same dimension; it is the scale that differs, and Spelf checks it.

```
type Impulse = Float<N*s, 0.0..=20000.0>;

fn total(nav: Impulse, ground: Float<lbf*s, 0.0..=4000.0>) -> Float<N*s, 0.0..=40000.0> {
  nav + convert<N*s>(ground)
}
```

Without `convert`, the sum is a type error. With it, the conversion factor comes from the unit definitions, not from a constant the programmer types, and the result range is computed through the conversion. Multiplication and division combine units: a gain in `N*m/rad` times an angle in `rad` is a torque in `N*m`. The compiler simplifies the units it prints (`s*rad/s` is shown as `rad`), but it compares them by dimension and scale, never by spelling.

The built-in units are m, km, ft, s, ms, us, min, h, kg, g, lb, N, lbf, Pa, rad, deg, Hz, kHz, MHz, A, T, uT, nT, B, KiB and MiB, and any product, quotient or integer power of them. A dimensionless value has unit `1`, which is also what you get when you write no unit at all.

Angles are a dimension of their own in Spelf, which is a departure from SI, where the radian is dimensionless. We made that choice because confusing an angle with a plain ratio is a real source of errors in attitude code, and because a degree is not the same as a radian in any useful sense for a programmer. The cost is an occasional explicit unit on a gain.

3.4 Reference frames and rotations

A vector component has a frame as well as a unit: `Float32<uT @ Body>` is a magnetic field component in the body frame; `Float<m/s @ ECI>` is a velocity in an Earth-centred inertial frame. Quantities in different frames cannot be added, compared or assigned to each other. Multiplying a framed quantity by a frameless one keeps the frame; multiplying two framed quantities is not allowed in the current core, because it would usually be part of a vector operation the core does not have yet.

Real systems need to change frames, and here the language has to give you a way that is both convenient and checkable. Spelf does it with a declared rotation:

```
struct MagV  { x: Float32<uT @ Mag>,  y: Float32<uT @ Mag>,  z: Float32<uT @ Mag> }
struct BodyV { x: Float32<uT @ Body>, y: Float32<uT @ Body>, z: Float32<uT @ Body> }

rotation MAG_TO_BODY: Mag -> Body = [[0.0, -1.0, 0.0], [1.0, 0.0, 0.0], [0.0, 0.0, 1.0]];

fn to_body(m: MagV) -> BodyV {
  rotate<BodyV>(MAG_TO_BODY, m)
}
```

The compiler checks that the matrix is a rotation: its rows are orthonormal to within 10^{-6} and its determinant is +1. That excludes three things that sometimes hide inside "mounting matrices": a scale factor (a calibration gain belongs in its own step, with its own unit), a skew, and a reflection. A

reflection is what you get when a sensor is mounted with the opposite handedness, and the sign correction for that is exactly the kind of fact that should not be buried in a matrix someone copied from an interface document.

`rotate` expands to explicit sums in a fixed order. Zero coefficients are skipped and coefficients of plus or minus one are exact (copy or negate), so a pure axis permutation introduces no rounding at all. General coefficients are rounded in the order written, and the C version of the same rotation, written by hand in the same order, gives the same bits. The rotation is directional: `rotate<MagV>(MAG_TO_BODY, b)` with `b` in the body frame is rejected.

A literal constant such as `0.0[uT]` has no frame of its own, and it takes the frame of whatever it is combined with or assigned to. That rule exists so that you can write `if v < 0.0[uT] { 0.0[uT] } else { v }` without ceremony. It applies only to literals; a variable without a frame stays frameless and cannot be added to a framed one, because nobody knows which frame it was measured in.

3.5 Time

Time instants have a clock: `Instant<TAI>`, `Instant<MET>`, `Instant<Monotonic>`. Instants on different clocks cannot be subtracted or compared. Deep Impact lost contact in 2013 when a time counter overflowed, and several missions have mixed mission elapsed time with an absolute time scale; in Spelf the subtraction does not type-check. Durations are floats with a time unit, so a timeout of `50[ms]` and a period of `0.1[s]` compare correctly.

3.6 Records, invariants and canonical forms

A `struct` groups fields and can carry a representation invariant:

```
struct ErrQ { w: Float<1, -4.0..=4.0>, x: Float<1, -4.0..=4.0>,
              y: Float<1, -4.0..=4.0>, z: Float<1, -4.0..=4.0> }
  invariant w >= 0.0
```

Every record value built anywhere in the program must satisfy the invariant at the point where it is built (rule R-505); a record that arrives as a parameter is assumed to satisfy it. The example is the canonical form of an attitude error quaternion. A quaternion and its negation represent the same rotation, but the sign of the error quaternion decides whether a controller turns the short way or the long way round. A controller that forgets to canonicalise works fine until the attitude estimator happens to flip the sign of its output, which estimators do, and then it commands a rotation of nearly 360 degrees. In our closed-loop test of a 6U CubeSat (section 10.3) the non-canonicalising controller loses 82 degrees of pointing at that moment.

Records can be state, as in `state bprev: BodyV = BodyV { x: 0.0[uT], y: 0.0[uT], z: 0.0[uT] };`; the initial value must be built from constants and must satisfy the invariant.

3.7 Fixed point

Some flight processors have no floating-point unit, and some algorithms are specified in fixed point. `Fix<16, 12, rad/s>` is a 16-bit fixed-point value with 12 fractional bits, in radians per second. Multiplication adds fractional bits and widens automatically; going back to a narrower format requires `requantize<T>(x, rounding = mode)`, and the rounding mode is mandatory: `nearest_even`, `toward_zero`, `floor` or `ceil`. There is no default because the direction of rounding in a feedback loop is a design decision (it decides whether you get a limit cycle), and an implicit default is how such decisions get made by accident. The four modes are implemented identically in the solver model, the interpreter and the generated C, and the three were checked against each other on 2256 cases.

4. Sensors and persistent state

4.1 Suspect values

A value read from a sensor is `Suspect<T>`. It has four variants, `Valid(v)`, `Saturated(v)`, `Stale(v)` and `Invalid`, and the only way to get at `v` is a `match` that handles every variant (rule R-402). A `Suspect<Float32<m>>` is not a `Float32<m>`: you cannot add it, compare it or pass it on.

This sounds like a small thing, and in a sense it is: in C the same information is a status field next to the value, and good code checks it. The difference is that in Spelf forgetting to check it is a compile error, so the check cannot be forgotten in the one place nobody tested. In the closed-loop lander test, the C version that reads the radar value while the status says invalid arms the touchdown detector at 200 m during a radar dropout, and a later burst of contact noise shuts the engines down at 132 m.

4.2 Provenance

Matching a `Suspect` gives you the value, but the compiler remembers where it came from. Every value derived from a sensor, by arithmetic, through a local variable, through a function's return value, or through a branch condition, carries a provenance mark. Two rules use it.

R-403: a sensor-derived value can be stored in persistent state only through `Held<T>`, which pairs it with a time stamp and a maximum age. Reading a `Held` value gives `Fresh(v)` or `Expired`, and both cases must be handled. The rule exists because the most common way for a bad measurement to cause damage is not to be used once, but to be stored and reused long after it should have been discarded. Schiaparelli's saturated attitude estimate persisted for about a second, which was enough.

```
state last: Held<Float<rad/s>> = held(0.0[rad/s], max_age = 50[ms]);

fn estimate(now: Instant<Monotonic>) -> Float<rad/s> {
  match sensor() {
    Valid(w) => { refresh(last, w, now); w }
    _ => match read(last, now) { Fresh(v) => v, Expired => 0.0[rad/s] },
  }
}
```

R-404: a sensor-derived condition can change a persistent boolean only through a `Latch`. `latch(persists_for = N)` declares that the condition must hold for N consecutive updates before the latch sets; a single false input resets the count; once set, the latch stays set until explicitly reset. The output of a latch is clean, which means you may base other persistent decisions on it.

Provenance follows control flow as well as data flow. `if sensor_value > 3.0 { flag = true; }` is a write to `flag` under a sensor-derived condition, and it is rejected exactly like `flag = sensor_value > 3.0`. Passing the value through a helper function does not launder it either; the tests `T-launder-local` and `T-launder-call` exist because the first implementation could be fooled that way.

4.3 Why this is a language feature and not a library

People who know SPARK sometimes ask why these rules need a new language. You can build a `Suspect` type in SPARK as a discriminated record, and we did, in a prototype that ships with the benchmark (`benchmark/spark-prototype`). It works for wrapping: the value cannot be read without checking the discriminant. What it cannot do is enforce provenance, because SPARK has no notion of "this value was derived from that one" across arbitrary computation, and it cannot make the use of the wrapper mandatory, because nothing stops the programmer from reading the sensor through a plain function returning a float. The rules that matter most here are rules about what the programmer is not allowed to do, and those need to live in the compiler.

5. Rules specific to control software

5.1 Saturation and integrator windup

`saturate(x, lo, hi)` does not return a number. It returns `Clamped(v)` when `x` was outside the limits and `Unclamped(v)` otherwise, and the program must say what happens in each case (rule R-407). The reason is integrator windup. A PI controller whose actuator sits at its limit keeps integrating an error it cannot correct; when the disturbance goes away, the stored integral drives a large overshoot. The usual fix is to freeze the integrator while the command is clamped, which requires knowing that it is clamped, which is exactly the information a plain `min(max(x, lo), hi)` throws away.

```
state integ: Float<rad*s, -2.0..=2.0> = 0.0[rad*s];

fn step(e: Float<rad, -1.0..=1.0>) -> Float<N*m, -0.5..=0.5> {
  let raw = KP * e + KI * integ;
  match saturate(raw, -0.5[N*m], 0.5[N*m]) {
    Unclamped(u) => {
      integ = integ + e * DT;
      u
    }
    Clamped(u) => u,
  }
}
```

There is a second, less obvious effect in this example. There is no clamp on the integrator. Because the integrator only runs when the command is not saturated, the compiler proves that it can never leave its declared range of plus or minus 2: if `integ` were already at 2, any error large enough to push it further would also saturate the command, and then the integrator would not run. If you add the usual defensive clamp, the compiler reports its branches as dead code (R-606). If you integrate in the `Clamped` arm as well, which is the windup bug, the range proof fails (R-501). So in this case the language catches the windup itself, not just the omission of the question.

That is not always so. If the integrator has no declared range, or if the programmer explicitly writes the integration into both arms, the compiler cannot know it is wrong. R-407 guarantees that the question is asked, not that it is answered correctly. In the fighter benchmark, the windup mutant written naturally (clamp, then integrate unconditionally) is rejected by R-407; in closed loop, the C version of the same mutant nearly triples the pitch-rate overshoot after a wind shear.

5.2 Redundancy and voting

Flight control computers read the same quantity from several sensors and vote. Spelf has declarations for the fault hypothesis (how many channels may fail, and how: by omission, by giving a wrong value, or arbitrarily) and for the redundancy architecture (channels, exchange rounds, authenticated or not). From these the compiler checks the classical bounds: three channels are enough for one value fault but not for one Byzantine fault without signatures, which needs four, and so on (rule R-412, after Lamport, Shostak and Pease, and Dolev and Strong). A function marked `@voter(G)` must read exactly the number of channels the architecture declares.

Tolerances in voters are another recurring problem: a fixed tolerance that was right for one sensor drift rate and one sampling period becomes wrong when either changes. R-413 requires the tolerance to be justified against declared drift and rate of change.

For a voter with persistent state, such as a filter that holds the last good value when channels disagree, `@output_bound` asks the compiler to check that, under the declared fault hypothesis and for arbitrary inputs from the faulty channels, the output stays within a bound of the true signal for a given number of steps (R-605). This is bounded model checking, and the horizon is part of the claim: the corrected Qantas 72 filter in our tests is proved for 8 steps and fails at 32, because with a permanently faulty channel the hold re-establishes itself. The assurance report states the horizon.

What none of this can do is choose the right voting algorithm. In the fighter closed-loop test, a controller that averages three gyros instead of taking the median loses control when one gyro fails hard-over. Spelf accepts that program, as do C, Rust and Ada, because averaging is a perfectly well-typed thing to do.

5.3 Validated envelopes

A function can declare the envelope it was validated for: `@valid_for(v_h <= 4000.0[m/s], validated_by = "Ariane 4 flight envelope")`. Every call site must prove that its arguments are inside the envelope (R-502). This is the root cause of Ariane 501 written as a rule: the inertial reference software had been validated for Ariane 4's horizontal velocities, and nothing stopped it from being called with Ariane 5's.

5.4 Modes, halting and safe paths

Flight software lives in modes, and some code must only run in some of them. A function declares the modes in which it may run, and a caller's mode set must be a subset of the callee's (05 §5.5). `halt` exists, but it must carry a justification and the list of modes in which it is allowed, and it is forbidden in the flight phase (R-401): Ariane 501's inertial reference system shut itself down on an exception during flight, which was the designed behaviour and the immediate cause of the loss.

A fault hypothesis can name devices that are assumed failed, and a function marked `@safe_path(H)` may not, through any chain of calls, touch a device that `H` says has failed (R-410). The compiler reports the call chain when it does. And a backup function declared with `@failover` is checked against its primary: if the two share code through the call graph, the compiler says so and the claim of independence has to be justified (R-405, R-411). Ariane 501's two inertial reference units ran identical software and failed identically, 72 milliseconds apart.

6. Concurrency and timing

6.1 Tasks and protected objects

A Spelf program can declare periodic tasks with a period, a priority, a worst-case execution time and a stack budget, and interrupt handlers with a vector, a level and a minimum inter-arrival time. State shared between tasks must live in a `protected` object; a plain `state` variable reached from two tasks, with at least one of them writing, is a data race and does not compile (03 §3.3). Protected objects use the immediate ceiling priority protocol, and the ceiling of each object is computed by the compiler from the tasks that use it. Writing a ceiling by hand is not allowed (R-302), because a hand-written ceiling that is too low is how Mars Pathfinder got its priority inversion.

```
protected Estimate {
  var rate: Int<-1000..=1000> = 0;
  fn put(r: Int<-1000..=1000>) effects { wcet <= 20[us] } { rate = r; }
  fn get() -> Int<-1000..=1000> effects { wcet <= 15[us] } { rate }
}

task Gnc(period = 10[ms], priority = 30, wcet = 3[ms]) { Estimate.put(5); }
task Tlm(period = 100[ms], priority = 10, wcet = 20[ms]) { let r = Estimate.get(); }
```

Nested protected calls are not allowed, which removes deadlock by construction.

6.2 Schedulability

With periods, priorities and execution times declared, the compiler runs response-time analysis (Joseph and Pandya, 1986) for every task, including the blocking a task can suffer on a protected object held by a lower-priority task, and including interrupts as sporadic load at the top of the priority range (R-304, R-306). A task set in which some task can miss its deadline is rejected, with the response time and the deadline in the message.

Mode changes are where this usually breaks in practice. Each mode maps to exactly one schedule; each schedule is analysed separately; and each mode transition is analysed as the union of the two schedules with worst-case parameters, because during the transition tasks of both may be active (R-305). A task that runs in no mode is reported as dead code. A degraded clock can be declared, `clock { nominal = 50[MHz], degraded = { 25[MHz] } }`, and then every analysis is repeated with execution times scaled to each clock (R-307). A watchdog declared with a timeout and a feeding task is checked in every scenario: the feeding task's period plus its response time must be less than the timeout, and the compiler checks that the task really calls the watchdog's feeding function.

The analysis rests on the declared WCETs. The compiler does not compute WCET; that needs a timing model of the processor, and it is future work. What it does do is make the dependence explicit: the schedulability claim is "given these execution times", and the times are in the source where a reviewer can see them.

6.3 Stack depth

Recursion is forbidden unless a function declares a bound, `recursive(n <= 8)` (R-201). The compiler computes the worst-case stack depth of each task from the call graph and the frame sizes in the assembly the C compiler actually produced for the target, not from an estimate at source level, and checks it against the task's declared stack budget. External functions declare their stack use as a contract.

7. Memory and devices

7.1 No heap, no surprises

There is no dynamic allocation in Spelf, no pointer type, no function pointers and no unbounded recursion. Arrays have a fixed length known at compile time and every index is proved in bounds. A bounded queue, `Bounded<T, N>`, has a `push` that returns `Ok` or `Full`, and the result cannot be ignored: the Spirit rover's flash file system problem on sol 18 began with a resource that filled up without anyone noticing. Return values in general cannot be silently dropped (the JPL rule 7).

Variables cannot be declared without an initial value. This removes uninitialised reads, which in C is one of the few defects every tool catches, and also removes the defensive initialisation to zero that hides the real question of what the value should be.

7.2 Resources with states

Some objects have a lifecycle that matters for safety: a DMA buffer that belongs either to the CPU or to the device, a sensor that must be configured before it is registered, a message slot that has been verified and must not be written. Spelf models these as linear resources with typestate (R-203). A resource type lists its states; a value of type `DmaBuf<Cpu>` is the buffer in the CPU state. Resources are linear: every value is consumed exactly once on every path, by moving it, by a declared state transition, by putting it back into its state variable, or by returning it. Using a resource after moving it, leaking it, consuming it inside a loop that does not put it back, taking it twice, and reaching its fields in a state that does not allow it are all compile errors.

```
region dma_ram { size = 1[KiB], kind = dma }
resource DmaBuf { Cpu, Device } in dma_ram device { Device } {
  words: [Int<0..=65535>; 32],
  count: Int<0..=32>,
}
resource DmaDone { Fired } { }
transition DmaBuf: Cpu -> Device;
transition DmaBuf: Device -> Cpu witness DmaDone;

fn transfer(b: DmaBuf<Cpu>) -> DmaBuf<Cpu> {
  let d = transit<Device>(b);
  bc_start(d);
  let done = bc_wait();
  transit<Cpu>(d, done)
}
```

States marked `device` cannot be touched by the CPU at all (R-205). The cache maintenance on each transition, clean and barrier when the buffer goes to the device, barrier and invalidate when it comes back, is derived by the compiler from the transition, not written by hand; on a target with coherent DMA (`--dma-coherent`) only the barriers remain. Taking the buffer back from the device requires a completion witness, a resource that only the device's completion path can produce. A verified state cannot be written (R-208), which is how a message checked once stays checked.

7.3 Memory regions

A `region` declares a block of memory with a size and a kind (`ram`, `dma` or `nvm`). Every resource placed in a region counts against its capacity, and in a `dma` region each instance is aligned to and padded to a cache line, so that cache maintenance on one buffer cannot corrupt its neighbour. The capacity check is a compile-time proof (R-209). `spelfc mem` then cross-checks the declared layout against the real object file produced for the target.

8. What the compiler generates

8.1 The C subset

The generated C is meant to be read. Each Spelf function becomes a C function with the same name prefixed by `f_`, each state variable a file-scope `static`, each constant a `static const`. Integer types are chosen from proved ranges, as described in section 3.1. Floating-point operations are written in the same order as in the source, which is what makes the cross-language comparisons in chapter 10 bit-exact. Sealed enumerations become `int32_t` tags. A `Suspect<T>` becomes a struct of a tag and a value. Arrays are passed as `const` pointers when they are not modified, and records and arrays are returned through an output pointer with `restrict`, which avoids hidden copies.

There are no run-time checks of any kind in the generated code, because there is nothing left to check: every condition a check would test has been proved.

8.2 Targets and substrates

With `--freestanding`, the generated C needs no standard library and no operating system: no `stdio`, no `main` unless you ask for one, and functions exported with external linkage for the flight application to call. We cross-compile it with GCC (the GNAT toolchains for ARM and RISC-V) and with Clang (through Zig) for three flight processors: the SPARC V8 LEON3 used on many European spacecraft, RISC-V RV32IMAC, and the ARM Cortex-R5F.

Two runtime substrates are included. The bare-metal one implements protected objects by masking interrupts up to the ceiling level and saving the previous mask on the caller's stack. The RTEMS 6 one maps tasks to rate-monotonic periods, protected objects to priority-ceiling semaphores, and DMA cache maintenance to `rtems_cache_*`. The RTEMS substrate is compiled against the RTEMS 6 Classic API signatures in our tests; it has not yet been run on a real RTEMS system.

8.3 Assurance levels

A module can declare the assurance level it is developed to with `@assurance(L0)` to `@assurance(L4)`. The level changes what the compiler requires. From L2 the IEEE floating-point model is mandatory. From L3, voters must carry an output bound and tasks must declare execution times and stack budgets. L4 is rejected outright, because at that level a qualified tool chain and verified WCET would be needed and the prototype provides neither. The levels are meant to map onto the design assurance levels of DO-178C and the criticality categories of ECSS, but the mapping is not formal and should not be read as a certification claim.

9. How the compiler proves things

9.1 Obligations

Every check described in the previous chapters is a proof obligation of the form: assumptions and path condition imply goal. The assumptions are the types of the parameters and state variables, the contracts of called functions, and the facts established earlier on the path. The goal is the property, for example that `hi * 256 + lo <= 65535`. The checker asks Z3 whether the negation of the goal is satisfiable together with the assumptions. "Unsatisfiable" means proved. "Satisfiable" means the solver found values for which the property fails, and those values become the error message. "Unknown" means the solver could not decide within the budget, and that is also a rejection (R-602), although, as described in section 3.2, for floating point the checker first tries to turn an unknown into a concrete counterexample.

State variables are treated as unknown values within their type at the start of every function, because the function can be called at any time. After a call to a function that writes some state, that state is unknown again. An early version of the checker did not do this and "proved" that a function returned 0 when it could return 10; the regression test for that bug, `T-callee-state-write`, is still in the suite.

Loops need invariants when they accumulate. `for i in 6..16 invariant s <= 255 * (i - 6) { ... }` states what is true at the start of every iteration; the checker proves it on entry and preserved by the body, and assumes it on exit. Without loop invariants the only way to prove an accumulation safe is a defensive check inside the loop, which is what we are trying to get rid of. This is the same experience SPARK users have with `Loop_Invariant`, and it is the main cost of the approach: the proofs are automatic, but the invariants are written by people.

9.2 Dead code

For every `if`, the checker asks two satisfiability questions: can the condition be true on this path, and can it be false? If one branch is unreachable, that is an error (R-606), because dead code in flight software is either a misunderstanding or a deactivated feature, and DO-178C wants both explained. A condition that depends only on a configuration constant is reported as deactivated code instead. This rule was what showed us, in the PI example of section 5.1, that the defensive integrator clamp could never fire.

9.3 The evidence bundle

`spelfc evidence` writes one SMT-LIB file per obligation, containing the assumptions and the negated goal, together with a manifest that indexes every obligation by rule, function and source line and records SHA-256 hashes of the source, of the generated C and of the compiler itself, and a short human-readable assurance summary. `tools/recheck_evidence.py` solves every file with Z3 and with cvc5, independently of the Spelf compiler. This catches a class of errors that the compiler cannot catch in itself: an optimisation in the checker that drops a needed fact. That is not a hypothetical. A cache in the checker that sliced the set of facts sent to the solver was keyed on Z3's internal object identifiers, which are reused after garbage collection; the independent re-check found a sliced query that was satisfiable, and the cache was fixed.

9.4 Why the solver budget is deterministic

Z3 can be given a time limit or a resource limit. Spelf always uses the resource limit (`rlimit`, 20 million by default), which counts internal solver steps, so the same obligation gets the same answer on every machine and every run. A certification artefact that depends on the speed of the machine that produced it is not an artefact anyone can reproduce.

10. Evidence

This chapter describes how we tested the claims in the rest of the guide. All of it can be rerun from the repository; the commands are in Appendix C. Every number here was produced by scripts that call real tools, and the raw data is committed next to the scripts.

10.1 Method

Three questions guided the evaluation. Does Spelf reject the defects it is designed to reject, and do the other languages? Does it cost anything at run time? And is the compiler itself right? A fourth question, whether engineers can use it productively, needs people other than us, and we have only prepared for it (section 10.7).

For the first question we used two kinds of programs. A corpus of 26 defect classes, each taken from a documented accident or incident, written as a faulty and a fixed program in every language; and five defect classes written into realistic flight controllers and flown in closed-loop simulation. For the second, seven flight workloads written with identical algorithms in four languages, compiled for three flight processors and executed in an emulator that counts instructions exactly. For the third, five independent checks on the compiler, described in section 10.6.

Every comparison is between languages used the way their best practitioners would use them, with the strongest freely available checker for each. A cell counts as a catch only if the faulty program is flagged and the fixed program is not. Without that second condition a compiler that rejects everything would score perfectly.

10.2 Twenty-six defects, six languages

The 26 defect classes range from unguarded narrowing (Ariane 501) and unit confusion (Mars Climate Orbiter) through priority inversion (Mars Pathfinder), stale sensor data (Schiaparelli), latched transients (Mars Polar Lander) and identical backup software (Ariane 501 again) to a second spike passing a filter at hold expiry (Qantas 72) and a Byzantine channel in a three-channel system with unsigned exchange. Each was written in C, MISRA C, Rust, Ada, SPARK and Spelf. The tools were GCC 16 with `-Wall -Wextra -Wconversion -fanalyzer` and a run under UndefinedBehaviorSanitizer, Cppcheck 2.20 with the official MISRA addon, rustc 1.98 with Clippy in pedantic mode and a run with overflow checks, GNAT 16.1 with all run-time checks and a run, and GNATprove 16.1 at level 2.

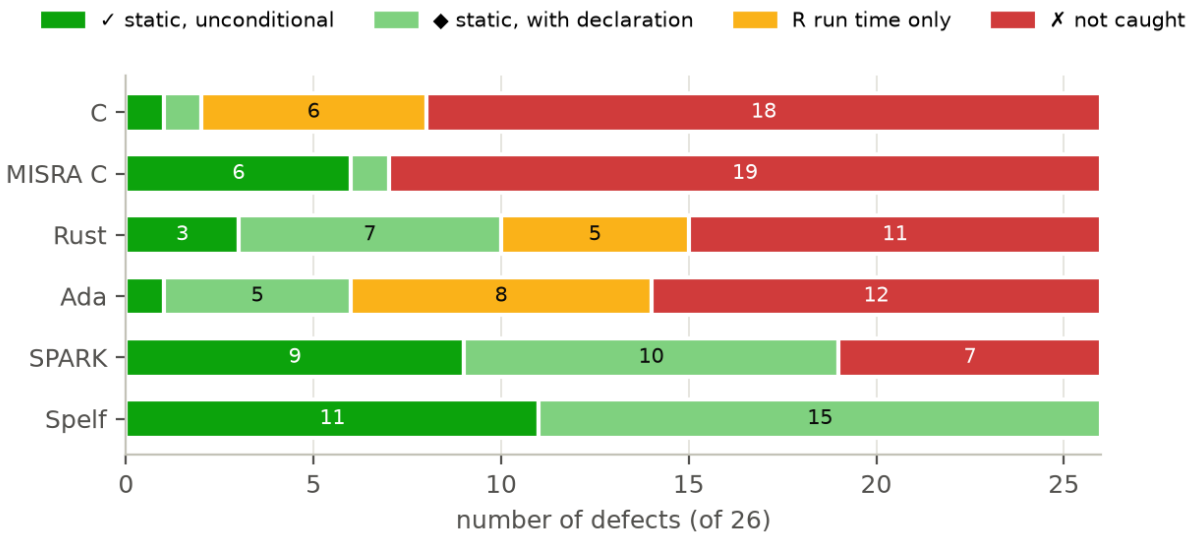


Figure 1. Outcome per language over the 26 accident-derived defect classes, from real tool output. "Static, with declaration" means the catch depends on a declaration the programmer could omit (a unit, a frame, a sensor source, an envelope); the same criterion is applied to Spelf.

Spelf rejects all 26 faulty programs and accepts all 26 fixed ones; 11 of the catches are unconditional and 15 depend on a declaration. SPARK catches 19, all statically. Rust catches 15, ten of them statically; Ada 14, six statically; C 8, two statically; MISRA C 7. The seven classes SPARK misses (stale data persisting, latched transients, identical backup code, a clamped command used blind, implicit fixed-point rounding, a Byzantine channel, and the hold-expiry spike) are all caught by Spelf.

Two things about this result should be kept in mind. The corpus was assembled by the people who designed Spelf, from the accidents Spelf was designed around, so it is biased in Spelf's favour; section 10.5 is our attempt to measure that bias. And the "with declaration" column matters: if the programmer omits the unit, Spelf does not catch the unit error, just as Rust and Ada would catch it only if the programmer had built unit types by hand. The honest summary is that Spelf turns these defects from "caught if someone was careful everywhere" into "caught if someone was careful once, at the declaration".



Figure 2. The full matrix. Each cell is the tool's verdict on the faulty program, counted only if the fixed program passes.

10.3 Complex systems in closed loop

The corpus programs are small. To see how the rules behave in realistic code, we wrote three onboard controllers in all four languages with identical algorithms and identical floating-point operation order, and flew them against physics models.

The first is a fly-by-wire pitch channel for a fighter with a statically unstable airframe (short-period model with positive pitch stiffness), running at 80 Hz. It votes triplex gyro and angle-of-attack sensors by median, schedules its gains on dynamic pressure with a piecewise-linear table, limits angle of attack, runs a PI law with anti-windup, and limits the elevator rate. The scenario has stick inputs, a gyro hard-over at 4 s, a wind shear from 9 to 12 s that saturates the elevator, and a stuck angle-of-attack vane at 16 s.

The second is B-dot detumbling for a 3U CubeSat at 10 Hz. The magnetometer is mounted turned by 90 degrees about the z axis, so its readings must be rotated into the body frame; the controller differentiates and filters the field and commands the magnetorquers, with logic for the first sample and for invalid samples. The physics is a rigid body with quaternion kinematics integrated by

fourth-order Runge-Kutta, in a geomagnetic field that rotates along the orbit, for one hour starting from a 15 degrees per second tumble.

The third is the terminal descent of a Mars lander at 100 Hz from 400 m: radar altitude with validity, a debounced touchdown detector armed below 40 m, and a safe descent rate when the radar drops out. The scenario has the leg-deploy shock at 2 s, a radar dropout at 9 s during which the radar reports invalid with a value of zero, and a burst of contact noise on two legs at 13 s.

Each controller was compiled in eight variants (C with GCC and with Clang, Ada with run-time checks, SPARK with checks suppressed after proof, Rust with and without overflow checks, Spelf through GCC and through Clang) for the three processors, linked with the same compiled physics model, and run in the Unicorn emulator. The same physics was also written in Python, with the controller run through the Spelf reference interpreter. All 54 runs (eight variants on two processors and two on LEON3, for three scenarios) produced trajectories bit-for-bit identical to the Python reference. Getting there required writing the floating-point operations in the same order everywhere, which is a useful discipline in itself.

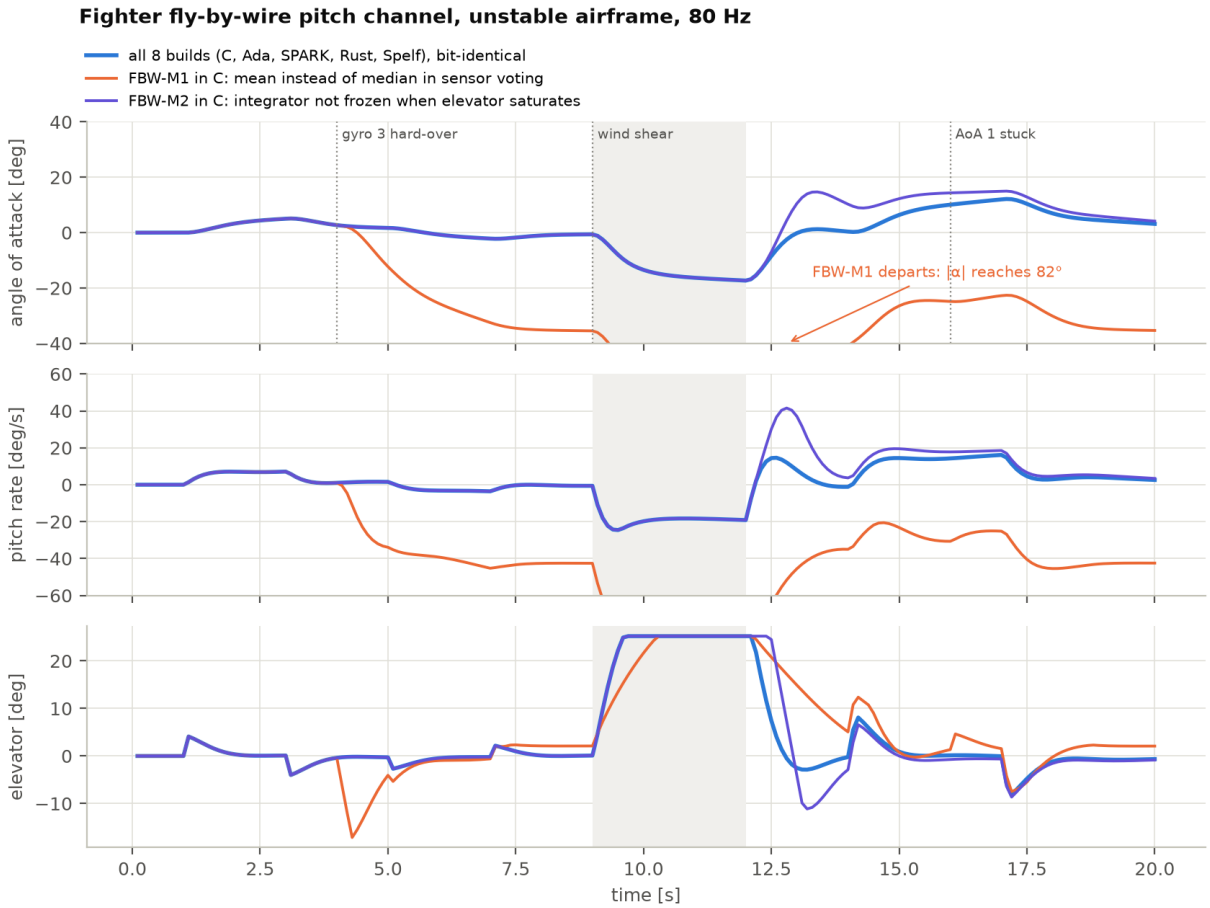


Figure 3. Fighter fly-by-wire pitch channel. Blue: all eight builds, identical. The wind shear saturates the elevator from 9 to 12 s.

Then we introduced five defects, each written the same way in all four languages, and flew the C version.

Defect	Effect in closed loop (C on RV32)	GCC	MISRA	Clippy	GNATprove	Spelf
Leg contact written straight to a flag (Mars Polar Lander)	engines cut at 350 m, impact at 55.6 m/s	accepted	accepted	accepted	proved free of RTE	rejected, R-404
Radar value used without its status	engines cut at 132 m, impact at 35.3 m/s	accepted	unused-member note	accepted	proved free of RTE	rejected, R-402
Magnetometer not rotated to body frame	4.85 deg/s after one hour, reference 0.23	accepted	accepted	accepted	proved free of RTE	rejected, 05 §5.2
Integrator not frozen at elevator saturation	pitch-rate overshoot 41.5 deg/s, reference 14.6	accepted	accepted	accepted	proved free of RTE	rejected, R-407
Mean instead of median in sensor voting	loss of control, angle of attack 82 deg	accepted	accepted	accepted	proved free of RTE	accepted

"Accepted" means the compiler and the analyser produced no diagnostic that the correct version did not also produce. The one exception is Cppcheck's note that the status field of the radar structure is never read, which is a real hint, though an indirect one. GNATprove is right in every row: none of these programs has a run-time error. They fail as control systems. That gap, between "no run-time errors" and "does the right thing with its sensors and its frames", is precisely where Spelf's provenance, frame and saturation rules sit.

The last row is in the table on purpose. Averaging three gyros instead of taking the median is a choice of algorithm, and no type system can know which algorithm you meant. With one gyro hard-over at plus 2 rad/s, the average is biased by two thirds of a radian per second and the aircraft departs. Spelf accepts the program, like every other language.

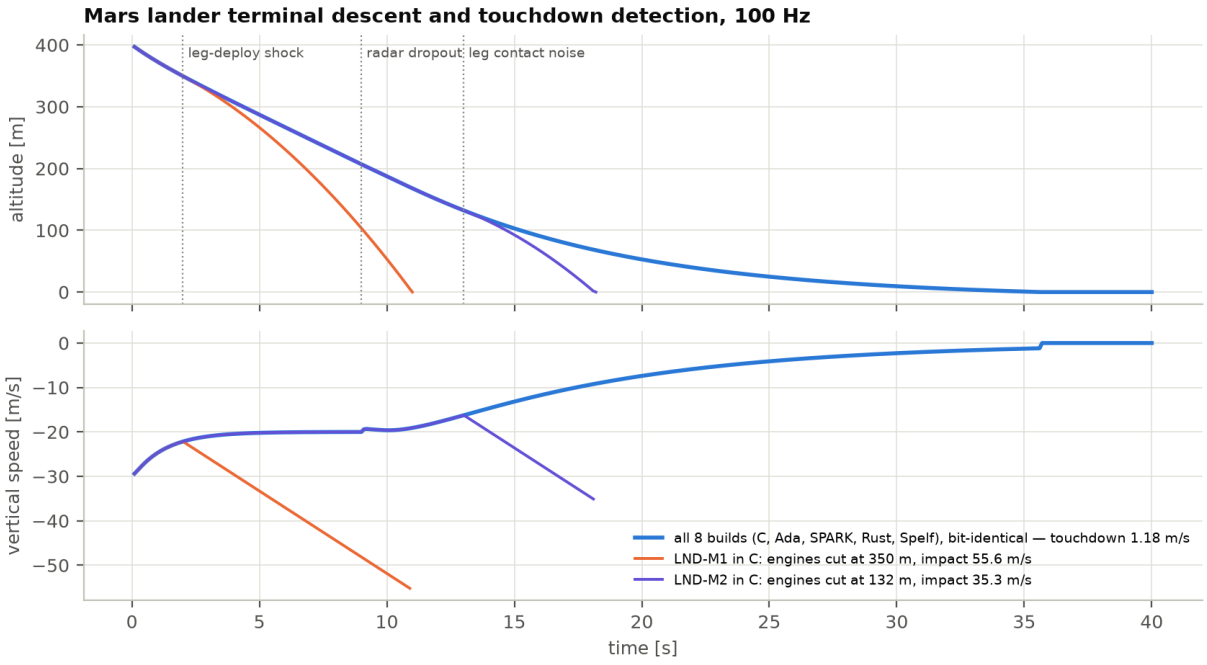


Figure 4. Mars lander. The Mars Polar Lander mutant cuts its engines at the leg-deploy shock; the Schiaparelli-style mutant arms its touchdown detector during the radar dropout and cuts its engines at the contact noise.

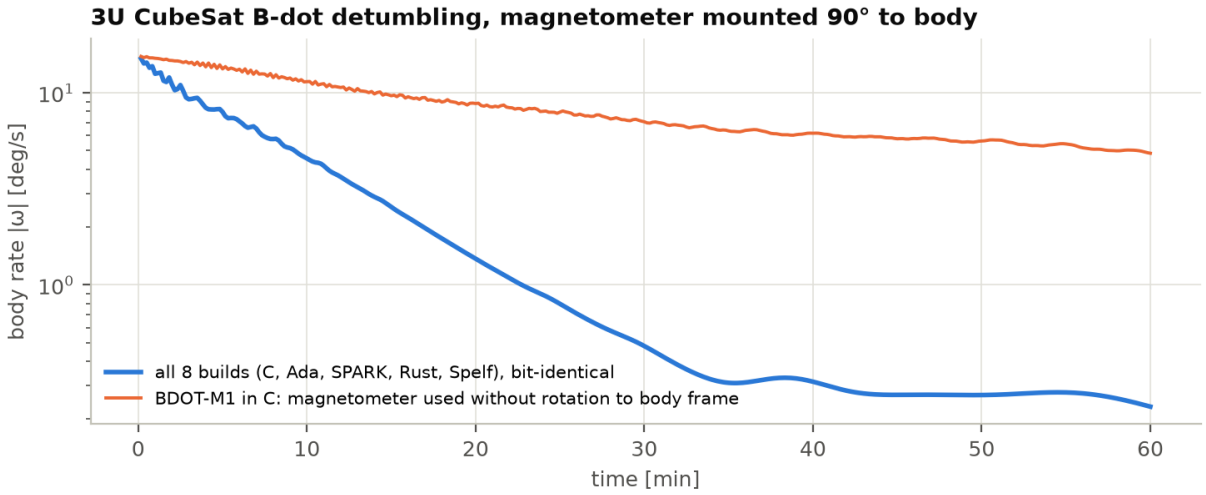


Figure 5. CubeSat detumbling. Using the magnetometer axes directly as body axes turns part of the B-dot torque into the wrong direction; after an hour the satellite is still tumbling at 4.85 degrees per second.

We also flew a fourth scenario from earlier work: a 6U CubeSat performing a 160-degree slew with a quaternion PD controller, during which the attitude estimator flips the sign of its quaternion at 300 s. All eighteen language and processor combinations matched the Python reference bit for bit; a controller without quaternion canonicalisation (rejected by Spelf with R-505) loses 82 degrees of pointing at the flip.

10.4 Cost

The seven workloads used for the performance comparison are the three controllers above, an ADCS attitude controller in double precision, a CCSDS space-packet primary-header decoder, a CRC-16 frame check over a 64-byte buffer, and the PX4 multicopter rate controller in single precision. Each was compiled with flight flags (`-O2`, freestanding, no unwind tables) and run on its inputs in the emulator, which counted the instructions executed per call exactly. All 126 cells (seven workloads, eight variants on two processors and two on the third) produced outputs bit-identical to the reference semantics.

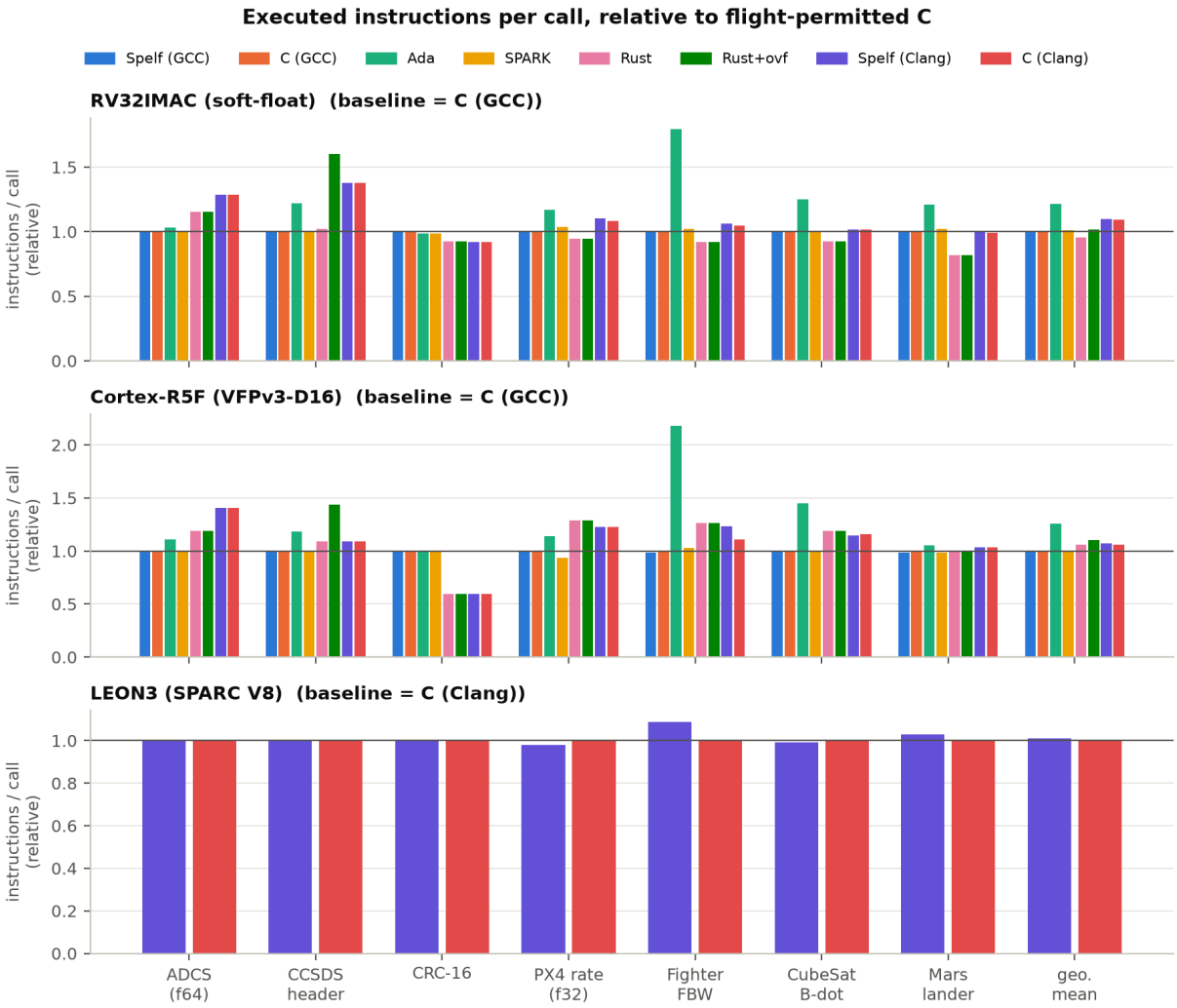


Figure 6. Instructions per call relative to flight C compiled with GCC (LEON3: relative to C with Clang, the only toolchain available for SPARC in our setup).

Spelf compared with	instructions per call	code size	run-time checks left
flight C, GCC	0.998	0.996	0 and 0
SPARK after proof, checks suppressed	0.996	0.944	0 and 0
Ada with run-time checks	0.806	0.662	0 and 34
flight C, Clang	1.010	1.001	0 and 0
Rust, release, no overflow checks	1.080	1.017	0 and 4
Rust with overflow checks	1.025	0.874	0 and 14

These are geometric means over all workloads and processors on which both variants ran. Against flight C and against proved SPARK, Spelf is level, within the noise of a few instructions per call. Against Ada with run-time checks it is about a fifth faster and a third smaller, because the checks Ada performs at run time are proved at compile time. Against Rust the picture needs explaining. Rust is ahead by 8 per cent on average, but the difference comes from the toolchain and not from the language: on RV32, which has no floating-point unit, rustc links its own software floating-point library, while every other build uses the `compiler-rt` library that comes with Zig, and C compiled with the same Clang gives exactly the same numbers as Spelf. On the Cortex-R5F, which has a

hardware FPU, Spelf through Clang is ahead of Rust on three workloads, level on two and behind on two, and the geometric mean there is about one per cent in Rust's favour.

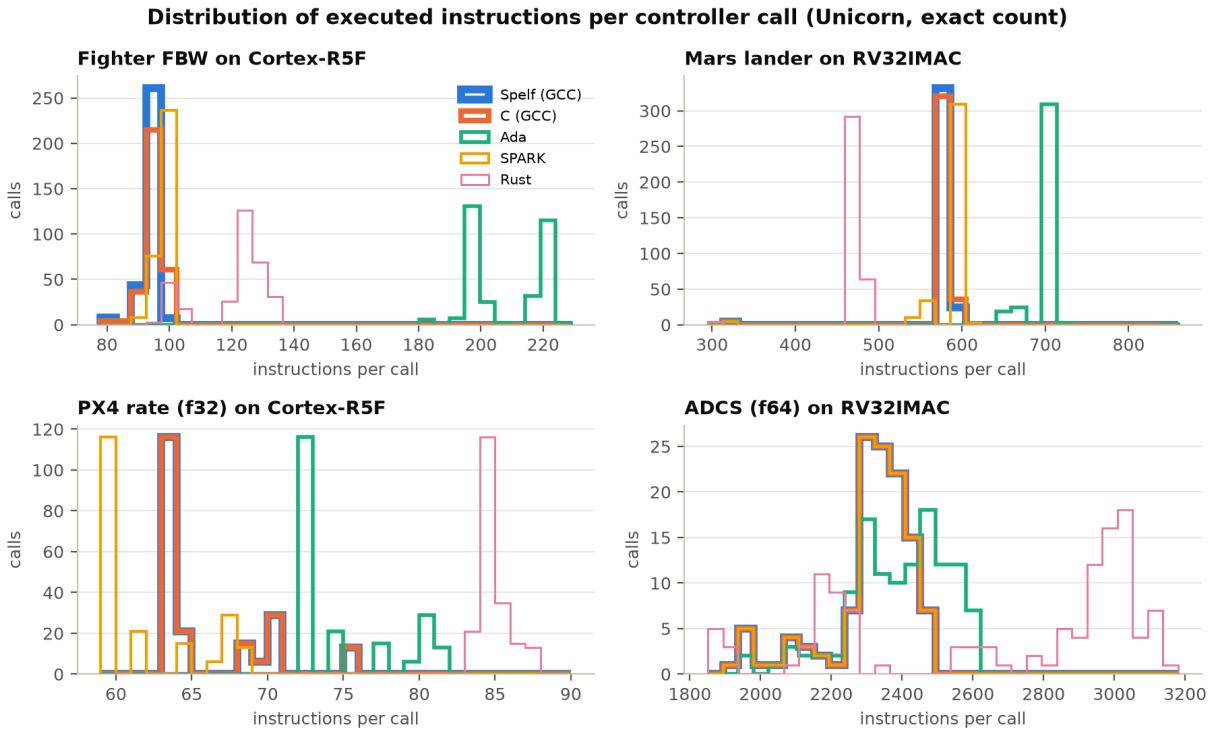


Figure 7. Distribution of instructions per call. Spelf (GCC), C (GCC) and SPARK often coincide; their histograms are drawn with decreasing line widths so that coincident ones stay visible.

Three caveats. Instruction counts are not execution times: the emulator models no caches and no pipeline. Spelf cannot be faster than unchecked C, since it produces C; the claim is that the proofs cost nothing at run time, not that Spelf code is faster. And the Spelf source is longer than the C source, by a factor of 1.06 to 2.06 in tokens on these workloads, because it carries units, ranges and contracts that the C does not have. The Ada/SPARK sources, which carry similar information as subtypes and contracts, are longer still.

10.5 Real bugs

The corpus is biased towards Spelf's design. To estimate the base rate, we took the 30 most recent merged pull requests in ArduPilot labelled BUG that fix flight code, without any selection by subject, and classified the root cause of each from its description.

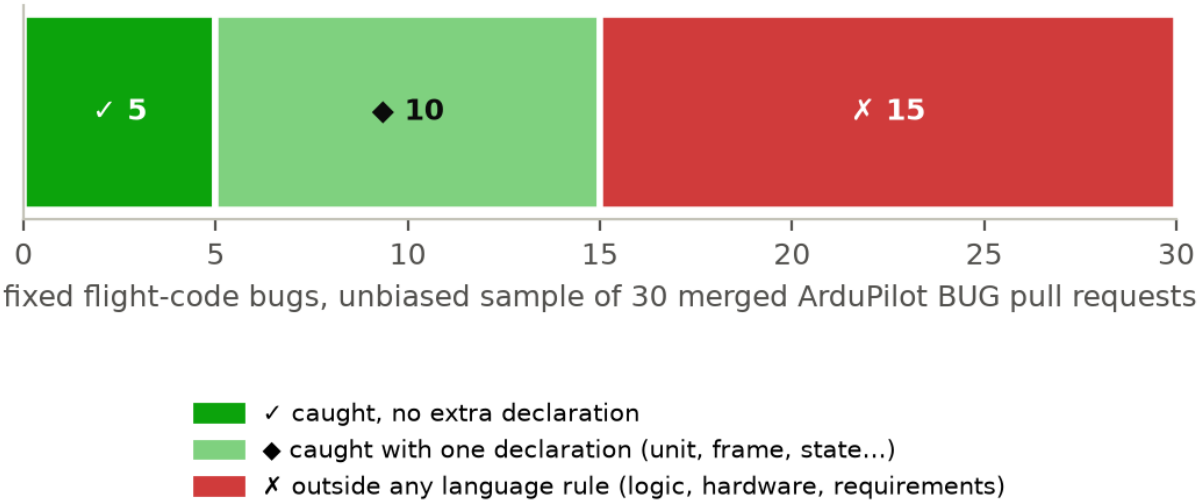


Figure 8. Unbiased sample of 30 fixed flight-code bugs from ArduPilot.

Five would have been rejected by a Spelf rule without any extra declaration (a null access, a stack overflow, a missing bounds check, a data race, an unchecked return value). Ten would have been rejected given one declaration: units in three cases, frames or datums in two, a wire encoding, a typestate, a configuration range, a freshness rule, and ownership of a memory-mapped register (a rule that is in the design but not yet in the compiler). Fifteen are logic, requirement or hardware errors that no language rule would catch. So half, and a sixth unconditionally. We think that is the honest headline: a language like Spelf removes a large and important class of bugs, the class that tends to destroy vehicles, and leaves the other half to testing, review and analysis as before.

A second, deliberately non-random sample of ten pull requests from PX4 and ArduPilot, chosen by keywords such as overflow, uninitialised, divide by zero and race, shows that the classes Spelf targets recur in both projects year after year. That sample measures recurrence, not base rate.

10.6 Is the compiler right?

A verifier that accepts a wrong program is worse than no verifier, because people stop looking. We check the prototype in five ways, and it is worth being precise about what each one establishes.

The soundness of the proof rules is mechanised in Z3. For the expression core, the argument is a structural induction checked step by step. For the full program logic there are 24 lemmas: loops with invariants, modular calls with state frames, protected objects under the ceiling protocol, interleavings of tasks, linear resources and others. Each lemma comes with a self-test that deliberately breaks the rule and must fail, so a lemma that is vacuously true is caught. What this does not establish is that the Python code implements the rules as stated; the derivation induction is a meta-argument, and a full mechanisation in Lean or Coq is future work.

Every proof obligation is exported to SMT-LIB and re-solved by cvc5, independently of the compiler. This catches errors in how the checker builds or simplifies queries.

A reference interpreter, independent of the code generator, runs every test program with a semantic oracle that checks at every step that each value is inside its static type. The interpreter's output is compared with the compiled C: 31 of 31 programs give identical output with no oracle violation.

The generated C is compiled for LEON3, RV32 and Cortex-R5, executed in the Unicorn emulator, and its output compared bit for bit with the interpreter: 31 programs on three processors, 93 of 93 identical, and in earlier runs 268 random programs and a 500-step sequence of the PX4 controller. This is binary-level translation validation for those programs; it says nothing about programs not run.

Soundness fuzzers generate random well-typed programs, compile them, and look for a property that was proved but fails at run time. One of these found a real soundness bug in the handling of single-precision range bounds, which were rounded to binary32 in the solver but left in double precision in the static range analysis; after the fix, 1500 programs gave 713 accepted and no violation.

Separately, the 143 accept and reject tests in `compiler/tests` cover every rule with at least one program that must be rejected and one that must be accepted, and `examples/mutations.json` holds 15 machine-checked claims of the form "make this change and the compiler rejects it with this rule".

10.7 Threats to validity, and what we would like others to do

The largest threat is that we wrote everything: the language, the corpus, the competitor programs, the classification of real bugs. We tried to write the C, Ada and Rust programs the way their best users would, but we cannot be sure we did. The repository contains a kit for independent evaluation: a shuffled list of the 30 ArduPilot pull requests without our verdicts, for blind classification; instructions for rewriting the competitor programs from scratch; and an invitation to make the C, Ada or Rust workloads faster while keeping their output bits. A result that contradicts ours is more useful to us than one that agrees.

Other threats are more ordinary. The emulator is not a processor. The workloads are small compared with a full flight application. The closed-loop models are simplified, although they are the standard textbook models. The real-bug sample is one project and thirty bugs.

11. Writing Spelf in practice

11.1 Start from the declarations

The work in Spelf moves to the declarations. Before writing a controller, write the types of its inputs and outputs with their units, frames and physical ranges: what the sensor can report, what the actuator can accept. Most of the proofs follow from those. Ranges should be physical, not defensive: the gyro's full scale, not "something large". A range that is too wide makes proofs fail for values that cannot occur; a range that is too narrow will be caught when the value is produced. Type aliases (`type Rate = Float32<rad/s, -35.0..=35.0>;`) keep signatures readable; they are abbreviations, not new types.

11.2 Reading a rejection

When the compiler rejects a program, the message has a rule identifier, a short description, and usually a counterexample: the input values that break the rule. The rule identifier is the most useful part. Appendix A says what each rule means. A counterexample is a concrete case you can reason about, which is more useful than a warning that something "may" overflow. If the counterexample cannot happen physically, the fix is usually to tighten a type; if it can, the fix is in the logic.

Some messages are about the solver, not the program. R-602 means the solver ran out of budget. That usually happens with long floating-point expressions that mix many variables with wide ranges; splitting the expression, narrowing a range, or factoring a helper function with a tight return type normally fixes it. We had this with the fighter controller's rate limiter. Written as "if the change exceeds the step, add the step to the previous value", the proof of the output range ran out of budget; written as "compute previous plus step and previous minus step, then choose", it was immediate, and it is also the clearer code.

11.3 Patterns that work

A few patterns came up repeatedly while writing the benchmark workloads.

For anti-windup, freeze the integrator in the `Clamped` arm and give the integrator state a range. As shown in section 5.1, the compiler can then often prove that the integrator stays in range without any clamp of its own.

For debouncing, use a `Latch` rather than a counter. It says what you mean, the compiler knows it is a debounce, and its output can drive persistent decisions.

For sensor redundancy, keep the voting function separate and give it a return type with the physical range of the quantity. Median of three is written with comparisons only and is exact in floating point, so it proves easily.

For frames, convert at the boundary. Rotate sensor readings into the body frame as soon as they are read, and keep everything inside the controller in one frame. The `rotate` call is then the only place where a mounting matrix appears, and it is checked.

For floating-point gains in a schedule, give the schedule function a return type with the range of the gains. That one range makes every product downstream provable.

11.4 Porting from C

We ported the PX4 multicopter rate controller from its C++ form. The port is line for line; the differences are the ones the language forces. The gains acquired units, and one constant, `radians(400)`, turned out to be a rate threshold of 400 degrees per second that had been written as an angle. The ranges of the inputs moved into their types. PX4's check that the integrator value is finite became unnecessary, because the compiler proves every intermediate value finite. The generated code is bit-identical to a PX4-style C implementation over 60 000 outputs, with 28 proofs, and its size is within minus 8 to plus 3 per cent of the C. The experience suggests that porting existing flight code is mostly a matter of writing down what the original code assumed, and that the assumptions are occasionally surprising.

12. Limits

Spelf is a research prototype, and it is important to be clear about what that means.

It does not prevent algorithm, requirement or hardware errors. Our best estimate, from the unbiased sample in section 10.5, is that about half of real flight bugs are of that kind.

It does not compute worst-case execution time. Schedulability is proved under declared execution times. A real WCET analysis needs a timing model of the processor, and that is future work.

The compiler is written in Python and is not qualified under DO-330 or ECSS-Q-ST-80. The evidence bundle is designed to support qualification, by letting an independent solver re-check every proof, but the path to a qualified tool is long and has not been started.

The soundness argument is mechanised for the rules but not for the implementation, and not end to end in an interactive theorem prover such as Lean or Coq. Response-time analysis rests on the published theorem rather than on a mechanised proof.

Some features specified in the design documents are not yet in the compiler: write-endurance budgets for non-volatile memory, ARINC 653 partitions with window budgets and inter-partition ports, memory-mapped I/O ownership. The RTEMS substrate compiles against the RTEMS 6 API but has not run on RTEMS.

Finally, every result in chapter 10 was produced by the people who designed the language. Until someone else reproduces or contradicts them, they should be read as a strong indication and not as an established fact.

Appendix A. Rule reference

Rule	What is checked
R-201	No recursion without a declared bound; worst-case stack depth from target assembly within each task's budget
R-202	Wire encodings: `decode` and `encode` are inverses over the whole wire range; a `Wire<E>` value is opened only by `decode`
R-203	Linear typestate: every resource value consumed exactly once on every path; transitions only as declared; field access by state
R-205	Device-owned states are not touched by the CPU; cache maintenance derived on each transition; completion witness to take a buffer back
R-208	A verified state cannot be written
R-209	Memory-region capacity, with cache-line alignment and padding in DMA regions
R-302	Ceiling priorities computed by the compiler; hand-written ceilings and nested protected calls forbidden
R-304	Response-time analysis with ceiling-protocol blocking
R-305	One schedule per mode; each schedule and each mode transition analysed
R-306	Interrupts: level, minimum inter-arrival time, sporadic load, race and ceiling analysis
R-307	Degraded clocks as separate scenarios; watchdog feeding interval below timeout in every scenario
R-401	`halt` needs a justification and allowed modes; forbidden in the flight phase
R-402	`Suspect<T>` opened only by a complete `match`
R-403	Sensor-derived values enter persistent state only through `Held<T>` with a maximum age
R-404	Sensor-derived conditions change persistent decisions only through a `Latch`
R-405, R-411	Backup and diversity claims checked against the call graph
R-407	`saturate` returns `Clamped` or `Unclamped`; both must be handled
R-410	Safe paths do not reach devices the fault hypothesis declares failed
R-412	Redundancy bounds for omission, value and Byzantine faults; voters read the declared channel count
R-413	Voter tolerances justified against drift and rate
R-501	Ranges, narrowing, division by zero, IEEE-754 finiteness and ranges
R-502	Calls stay inside declared validation envelopes
R-505	Representation invariants hold for every value built
R-602	An obligation the solver cannot decide is not a proof
R-605	Bounded output error under a fault hypothesis, for a stated horizon
R-606	Unreachable branches and statements are errors; configuration-dependent ones are reported as deactivated
02 §2.5	Array indices in bounds
03 §3.3	No data races between tasks; shared state only in protected objects
03 §3.5	Instants on different clocks do not mix
05 §5.1	Fixed-point rescaling only with an explicit rounding mode; binary32 and binary64 do not mix implicitly
05 §5.2	Units (dimension and scale) agree; frames agree; frame changes only through a checked rotation
05 §5.5	A caller's modes are a subset of the callee's
05 §5.7	Configuration and constant values are computable at compile time and in range
06 §6.2	Assurance level requirements (L2: IEEE model; L3: output bounds, WCET and stack; L4 rejected)

Appendix B. Grammar summary

This is a condensed version of the grammar that covers everything used in this guide. The parser (`compiler/spelfc/parser.py`) is the reference.

```
program     = { item } ;
item        = module | mode_lattice | flight_phase | enum | struct | const | config | state
            | encoded | fault_hypothesis | redundancy | task | isr | protected | fn | extern_fn
            | schedule | clock | watchdog | resource | transition | region | rotation | type_alias ;

type        = "Int" "<" int "..=" int ">"
            | ( "Float" | "Float32" ) "<" unit [ "@" FRAME ] [ "," num "..=" num ] ">"
            | "Fix" "<" bits "," frac "," unit ">"
            | "Instant" "<" CLOCK ">" | "bool" | "[" type ";" N "]"
            | "Suspect" "<" type ">" | "Held" "<" type ">" | "Latch" | "Bounded" "<" type "," N ">"
            | "Wire" "<" ENCODING ">" | RESOURCE "<" STATE { "|" STATE } ">" | "&" RESOURCE "<" STATE ">"
            | NAME ;

struct      = "struct" NAME "{" field { "," field } "}" [ "invariant" expr ] ;
rotation    = "rotation" NAME ":" FRAME "->" FRAME "=" "[" row "," row "," row "]" ";" ;
fn          = { annotation } "fn" NAME "(" params ")" [ "->" type ] { clause } block ;
clause      = "requires" expr | "ensures" expr | "effects" "{" effect { "," effect } "}"
            | "recursive" "(" NAME "<=" INT ")" ;
stmt        = "let" NAME [ ":" type ] "=" expr ";" | "var" NAME ":" type "=" expr ";"
            | lvalue "=" expr ";" | "if" expr block [ "else" block ]
            | "for" NAME "in" INT ".." INT [ "invariant" expr ] block
            | "return" [ expr ] ";" | "halt" "(" "because" "=" STRING "," "allowed_in" "=" "{" modes "}" ")" ";"
            | expr ";" ;
expr        = literal [ "[" unit "]" ] | NAME | NAME "::" VARIANT | NAME "." FIELD | call | match | if
            | NAME "{" field_init { "," field_init } "}"
            | "narrow" "<" type ">" "(" expr ")" | "convert" "<" unit ">" "(" expr ")"
            | "requantize" "<" type ">" "(" expr "," "rounding" "=" MODE ")"
            | ( "wire" | "encode" ) "<" ENCODING ">" "(" expr ")"
            | "rotate" "<" STRUCT ">" "(" ROTATION "," expr ")"
            | "transit" "<" STATE ">" "(" expr [ "," witness ] ")" | binary and unary operators ;
```

Built-in functions: `print`, `saturate`, `decode`, `feed`, `is_set`, `reset`, `held`, `latch`, `refresh`, `read`, `push`, `empty`, `bits`, `take`, `put`, `xor`, `band`, `bor`, `shl`, `shr`. These names are reserved and cannot be redefined, so that a call always means the same thing.

Appendix C. Reproducing the results

The tools and versions we used are listed in the README. With them installed, the following commands regenerate every number and figure in chapter 10. The first two blocks take a few minutes; the spacecraft benchmark takes about fifteen, most of it in GNATprove.

```
cd compiler
python tests/run_tests.py
python tests/diff_test.py
python tests/tools_test.py
python tools/xval_unicorn.py --tests
python ../examples/check_examples.py

cd ../benchmark/multilang
python run.py
python report.py

cd ../spacecraft
python run.py
python sil.py
python sil_complex.py
python report.py

cd ../figures
python make_figures.py
```

The raw results are written to `results.json` files next to each script, and the figures to `benchmark/figures`. Nothing in the figures or in this guide was edited by hand after generation, apart from the prose.